Vidyavardhini's
College of Engineering & Technology
Vasai Road (W)

Department of Artificial Intelligence & Data Science Engineering

Laboratory Manual
Student Copy

Semester	III(NEP)	**Class**	**S.E.**
Course Code	2483115		
Course Name	Analysis of Algorithms Lab		
Academic Year	2026-27		

Vidyavardhini's College of Engineering & Technology

Vision

To be a premier institution of technical education; always aiming at becoming a valuable resource for industry and society.

Mission

- To provide technologically inspiring environment for learning.
- To promote creativity, innovation and professional activities.
- To inculcate ethical and moral values.
- To cater personal, professional and societal needs through quality education.

Department Vision:

To foster proficient artificial intelligence and data science professionals, making remarkable contributions to industry and society.

Department Mission:

- To encourage innovation and creativity with rational thinking for solving the challenges in emerging areas.
- To inculcate standard industrial practices and security norms while dealing with Data.
- To develop sustainable Artificial Intelligence systems for the benefit of various sectors.

Program Specific Outcomes (PSOs):

PSO1: Analyse the current trends in the field of Artificial Intelligence & Data Science and convey their finding by presenting / publishing at a national / international forums.

PSO2: Design and develop Artificial Intelligence & Data Science based solutions and applications for the problems in the different domains catering to industry and society.

Program Outcomes (POs):

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO8. Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO9. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO10. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO11. Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Objective

1	To introduce the methods of designing and analysing algorithms
2	Design and implement efficient algorithms for a specified application
3	Strengthen the ability to identify and apply the suitable algorithm for the given real-world problem.
4	Analyse worst-case running time of algorithms and understand fundamental algorithmic problems.

Course Outcomes

At the end of the course student will be able to:		**Action verbs**	**Bloom's Level**
2483115.1	Analyze time complexity of sorting algorithms	Analyze	Apply (Level 3)
2483115.2	Analyze the complexity of problems solved using divide and conquer approaches	Analyze	Apply (Level 3)
2483115.3	Implement greedy algorithms for solving Dijkstras, Minimum spanning tree & fractional knapsack.	Implement	Apply (Level 3)
2483115.4	Implement dynamic programming algorithm for All pair shortest path and 0/1 knapsack	Apply	Apply (Level 3)
2483115.5	Implement backtracking and branch and bound for 15 puzzle, N queen and sum of subset problem	Apply	Apply (Level 3)
2483115.6	Analyze the performance of string-matching techniques	Analyze	Apply (Level 3)

Mapping of Experiments with Course Outcomes

Sr. No	Title	24831 15.1	2483 115.2	24831 15.3	24831 15.4	24831 15.5	2483 115.6
1.	To implement Insertion Sort and Selection Sort with comparative analysis for large values of 'n'.	3	-	-	-	-	-
2.	To implement Binary Search for 'n' numbers and perform analysis using DAC Technique.	-	3	-	-	-	-
3.	To implement Quick Sort and perform comparative analysis for large values of 'n' using DAC technique.	-	3	-	-	-	-
4.	To implement Fractional Knapsack using Greedy Method.	-	-	3	-	-	-
5.	To implement Prim's MST Algorithm using Greedy Method.	-	-	3	-	-	-
6.	To implement All Pairs Shortest Path Algorithm using Dynamic Method (Floyd Warshall).	-	-	-	3	-	-
7.	To implement Sum of Subset Algorithm using Backtracking Method	-	-	-	-	3	-
8.	To implement N-queen Algorithm using Backtracking method	-	-	-	-	3	-
9.	To implement the Rabin-Karp Algorithm for String Matching and analyze its time complexity.	-	-	-	-	-	3
10.	To implement the Min-Max Algorithm.	-	3	-	-	-	-

Enter correlation level 1, 2 or 3 as defined below
1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)

If there is no correlation put "— ".

List of Experiments

Sr. No	Name of Experiments	DOP	DOC	Marks	Sign
1.	To implement Insertion Sort and Selection Sort with comparative analysis for large values of 'n'.				
2.	To implement Binary Search for 'n' numbers and perform analysis using DAC Technique.				
3.	To implement Quick Sort and perform comparative analysis for large values of 'n' using DAC technique.				
4.	To implement Fractional Knapsack using Greedy Method.				
5.	To implement Prim's MST Algorithm using Greedy Method.				
6.	To implement All Pairs Shortest Path Algorithm using Dynamic Method (Floyd Warshall).				
7.	To implement Sum of Subset Algorithm using Backtracking Method				
8.	To implement N-queen Algorithm using Backtracking method				
9.	To implement the Rabin-Karp Algorithm for String Matching and analyze its time complexity.				
10.	To implement the Min-Max Algorithm.				
Assignments					
1.	Assignment 1: Sorting Algorithms				
2.	Assignment 2: Divide and Conquer Approach				
3.	Assignment 3 : Greedy Method Approach				
4.	Assignment 4 : Dynamic Programming Approach				

5.	Assignment 5: Backtracking and Branch and Bound				
6.	Assignment 6: String Matching Algorithms				
Formative Assessment					
1	TH- Quiz 1: Sorting Algorithms				
2	TH- Quiz 2: Divide and Conquer Approach				
3	TH- Quiz 3: Greedy Method Approach				
4	TH- Quiz 4: Dynamic Programming Approach				
5	TH- Quiz 5: Backtracking and Branch and Bound				
6	TH - Quiz 6: String Matching Algorithms				
7	PR - Quiz 1: Insertion and Selection Sort				
8	PR- Quiz 2: Divide and Conquer Approach				
9	PR- Quiz 3: Fractional Knapsack ad Job Sequencing, MST				
10	PR- Quiz 4: 0\1 Knapsack, TSP, All Pair Shortest Path algorithm				
11	PR- Quiz 5: N-Queen and 15 Puzzle Problem				
12	TH - Quiz 6: Naïve String , KMP and Rabin Karp				

D.O.P: Date of performance

D.O.C : Date of correction

Experiment No.1
Insertion Sort and Selection Sort
Date of Performance:
Date of Submission:

Experiment No. 1

Title: Insertion Sort and Selection Sort

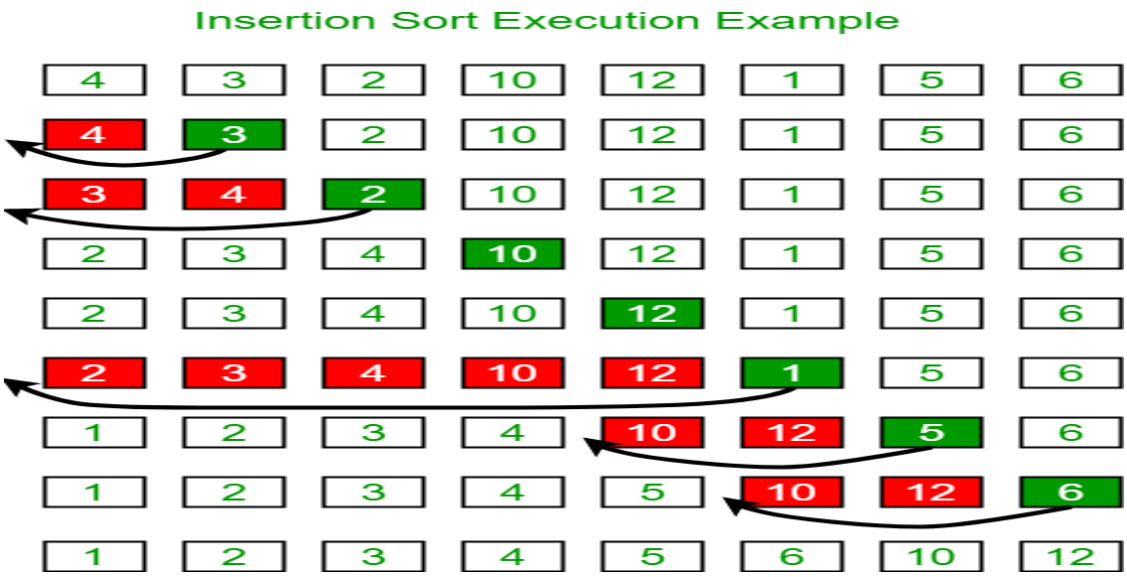
Aim: To implement Insertion and Selection sort with Comparative analysis for large values of 'n'

Objective: To introduce the methods of designing and analysing algorithms

Theory:

Insertion sort is a simple sorting algorithm that works similarly to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Example:



Algorithm and Complexity:

INSERTION-SORT(A)	*cost*	*times*
1 **for** $j = 2$ **to** $A.length$	c_1	n
2 $key = A[j]$	c_2	$n-1$
3 // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
4 $i = j-1$	c_4	$n-1$
5 **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n}(t_j-1)$
7 $i = i-1$	c_7	$\sum_{j=2}^{n}(t_j-1)$
8 $A[i+1] = key$	c_8	$n-1$

Selection sort is a sorting algorithm, specifically an in-place comparison sort. Selection sort is noted for its simplicity, and it has performance advantages over more complicated algorithms in certain situations, particularly where auxiliary memory is limited.

The algorithm divides the input list into two parts: the sub list of items already sorted, which is built up from left to right at the front (left) of the list, and the sub list of items remaining to be sorted that occupy the rest of the list. Initially, the sorted sub list is empty, and the unsorted sub list is the entire input list. The algorithm proceeds by finding the smallest (or largest, depending on sorting order) element in the unsorted sub list, exchanging it with the leftmost unsorted element (putting it in sorted order), and moving the sub list boundaries one element to the right.

Example:

arr[] = 64 25 12 22 11

// Find the minimum element in arr[0...4] // and place it at beginning

11 25 12 22 64

// Find the minimum element in arr[1...4] // and place it at beginning of arr[1...4]

11 12 25 22 64

// Find the minimum element in arr[2...4] // and place it at beginning of arr[2...4]

11 12 **22** 25 64

// Find the minimum element in arr[3...4] // and place it at beginning of arr[3...4]

11 12 22 **25** 64

Algorithm and Complexity:

Alg.: SELECTION-SORT(A)		
	cost	Times
n ← length[A]	c_1	1
for j ← 1 to n - 1	c_2	n-1
do smallest ← j	c_3	n-1
for i ← j + 1 to n	c_4	$\sum_{j=1}^{n-1}(n-j+1)$
≈ n2/2 comparisons, do if A[i]<A[smallest]	c_5	$\sum_{j=1}^{n-1}(n-j)$
then smallest ← i	c_6	$\sum_{j=1}^{n-1}(n-j)$
≈ n exchanges, exchange A[j] ↔ A[smallest]	c_7	n-1

Implementation:

Conclusion:

Experiment No. 2
Binary Search Algorithm
Date of Performance:
Date of Submission:

Experiment No. 2

Title: Binary Search Algorithm

Aim: To study and implement Binary Search Algorithm

Objective: To introduce Divide and Conquer based algorithms

Theory:

Search a sorted array by repeatedly dividing the search interval in half. Begin with an interval covering the whole array. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half. Otherwise, narrow it to the upper half. Repeatedly check until the value is found or the interval is empty

- Binary search is efficient than linear search. For binary search, the array must be sorted, which is not required in case of linear search.
- It is divide and conquer based search technique.
- In each step the algorithms divides the list into two halves and check if the element to be searched is on upper or lower half the array
- If the element is found, algorithm returns.

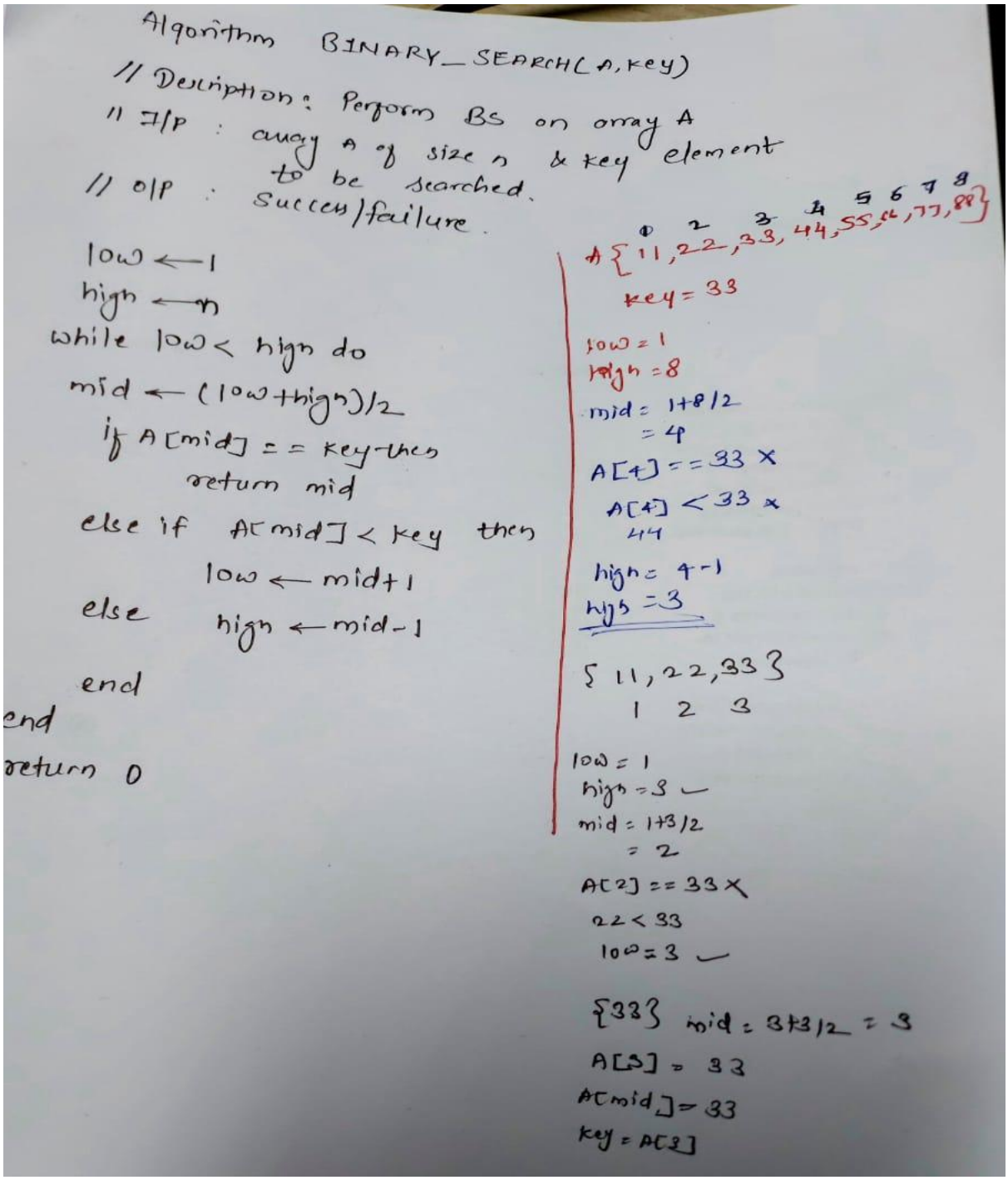


The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(Log n).

- Compare x with the middle element.
- If x matches with the middle element, we return the mid index.
- Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we recur for the right half.
- Else (x is smaller) recur for the left half.
- Binary Search reduces search space by half in every iterations. In a linear search, search space was reduced by one only.
- n=elements in the array
- Binary Search would hit the bottom very quickly.

	Linear Search	Binary Search
2nd iteration	n-1	n/2
3rd iteration	n-2	n/4

Example:


Algorithm BINARY_SEARCH(A, key)
// Description: Perform BS on array A
// I/P : array A of size n & key element to be searched.
// O/P : Success/failure.
low ← 1
high ← n
while low < high do
mid ← (low+high)/2
if A[mid] == key then
return mid
else if A[mid] < key then
low ← mid+1
else
high ← mid-1
end
end
return 0
A { 11, 22, 33, 44, 55, 66, 77, 88 }
0 1 2 3 4 5 6 7 8
key = 33
low = 1
high = 8
mid = 1+8/2
= 4
A[4] == 33 ×
A[4] < 33 ×
44
high = 4-1
high = 3
{ 11, 22, 33 }
1 2 3
low = 1
high = 3
mid = 1+3/2
= 2
A[2] == 33 ×
22 < 33
low = 3
{33} mid = 3+3/2 = 3
A[3] = 33
A[mid] = 33
key = A[3]

Algorithm and Complexity:

The binary search

- Algorithm 3: the binary search algorithm

Procedure binary search (x: integer, a_1, a_2, …,a_n: increasing integers)
  i :=1 { i is left endpoint of search interval}
  j :=n { j is right endpoint of search interval}
While i < j
begin
    m := $\lfloor (i + j) / 2 \rfloor$
    if x > a_m **then** i := m+1
    else j := m
end
If x = a_i **then** *location* := i
else *location* :=0
{*location* is the subscript of the term equal to x, or 0 if x is not found}

2

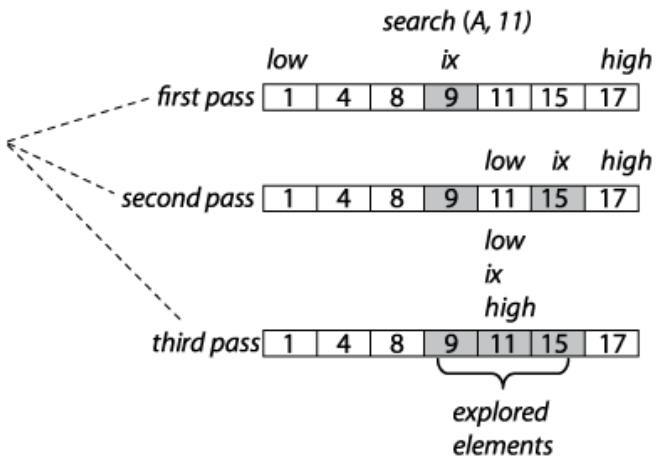
BINARY SEARCH

Best	Average	Worst
O (1)	O (log n)	O (log n)

Array

Divide and Conquer

```
search (A, t)
1.   low = 0
2.   high = n –1
3.   while (low ≤ high) do
4.     ix = (low + high)/2
5.     if (t = A[ix]) then
6.         return true
7.     else if (t < A[ix]) then
8.         high = ix – 1
9.     else low = ix + 1
10.  return false
end
```



Best Case:
Key is first compared with the middle element of the array.
The key is in the middle position of the array, the algorithm does only one comparison, irrespective of the size of the array.
T(n)=1

Worst Case:
In each iteration search space of BS is reduced by half, Maximum log n(base 2) array divisions are possible.
Recurrence relation is
T(n)=T(n/2) +1
Running Time is O(logn).

Average Case:

Key element neither is in the middle nor at the leaf level of the search tree.
It does half of the log n(base 2).
Base case=O(1)
Average and worst case=O(logn)

Implementation:

Conclusion:

Comment on the implementation of Binary Search Algorithm.

Experiment No.3
Quick Sort
Date of Performance:
Date of Submission:

Experiment No. 3

Title: Quick Sort

Aim: To implement Quick Sort and Comparative analysis for large values of 'n'.

Objective: To introduce the methods of designing and analyzing algorithms.

Theory:

The merge sort algorithm closely follows the divide-and-conquer paradigm. Intuitively, it operates as follows:

1. Divide: Divide the n-element sequence to be sorted into two subsequences of n=2 elements each.
2. Conquer: Sort the two subsequences recursively using merge sort.
3. Combine: Merge the two sorted subsequences to produce the sorted answer.

Partition-exchange sort or quicksort algorithm was developed in 1960 by Tony Hoare. He developed the algorithm in order to sort the words to be translated, to make them more easily matched to an already-sorted Russian-to-English dictionary that was stored on magnetic tape.

Quick sort algorithm on average, makes O(n log n) comparisons to sort n items. In the worst case, it makes O(n2) comparisons, though this behavior is rare. Quicksort is often faster in practice than other O(n log n) algorithms. Additionally, quicksort's sequential and localized memory references work well with a cache. Quicksort is a comparison sort and, in efficient implementations, is not a stable sort. Quicksort can be implemented with an in-place partitioning algorithm, so the entire sort can be done with only O(log n) additional space used by the stack during the recursion.

Quicksort is a divide and conquer algorithm. Quicksort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quicksort can then recursively sort the sublists.

1. Elements less than pivot element.

2. Pivot element.

3. Elements greater than pivot element.

Where pivot as middle element of large list. Let's understand through example:

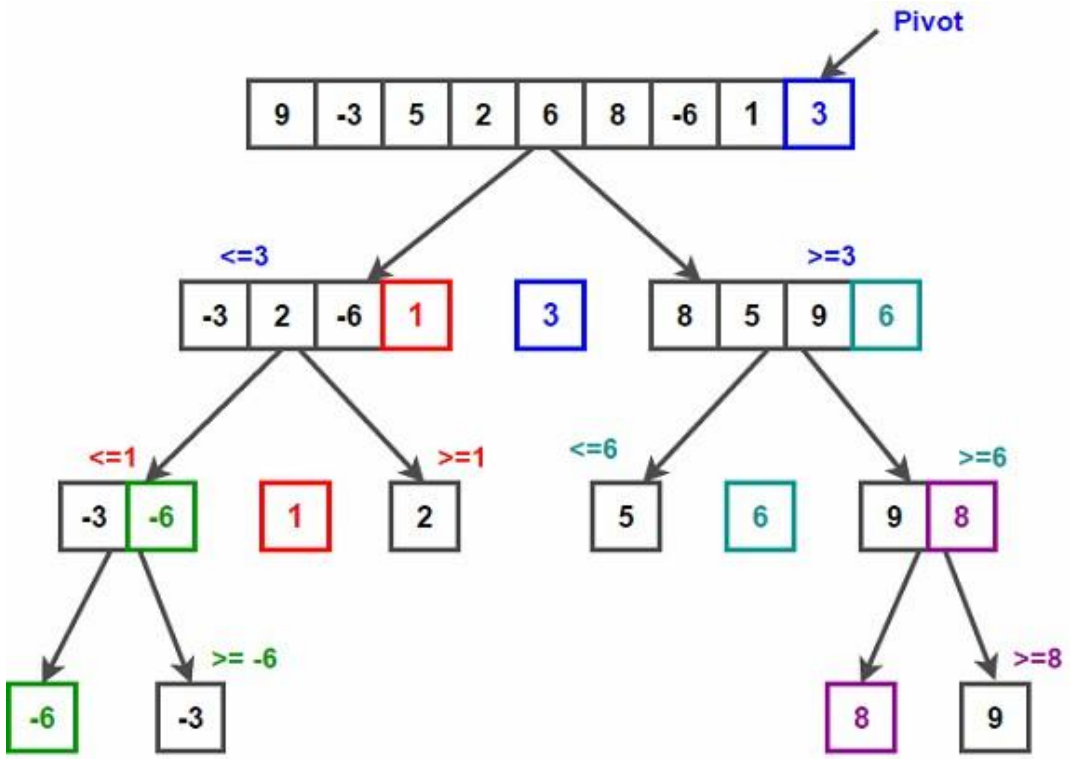
List : 3 7 8 5 2 1 9 5 4

In above list assume 4 is pivot element so rewrite list as:

3 1 2 4 5 8 9 5 7

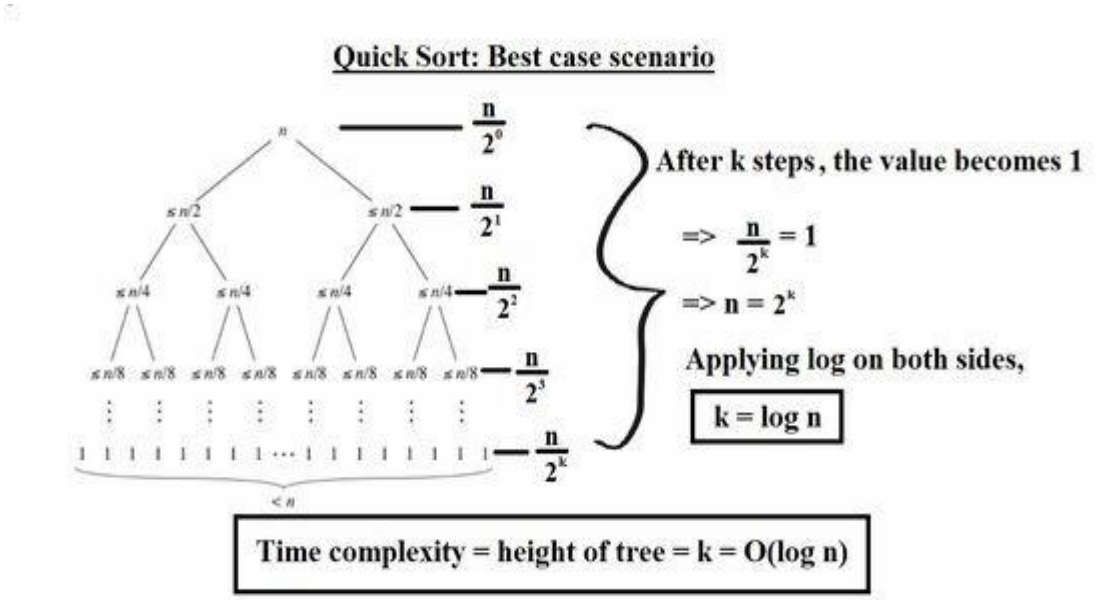
Here, I want to say that we set the pivot element (4) which has in left side elements are less than and right hand side elements are greater than. Now you think, how's arrange the less than and greater than elements? Be patient, you get answer soon.

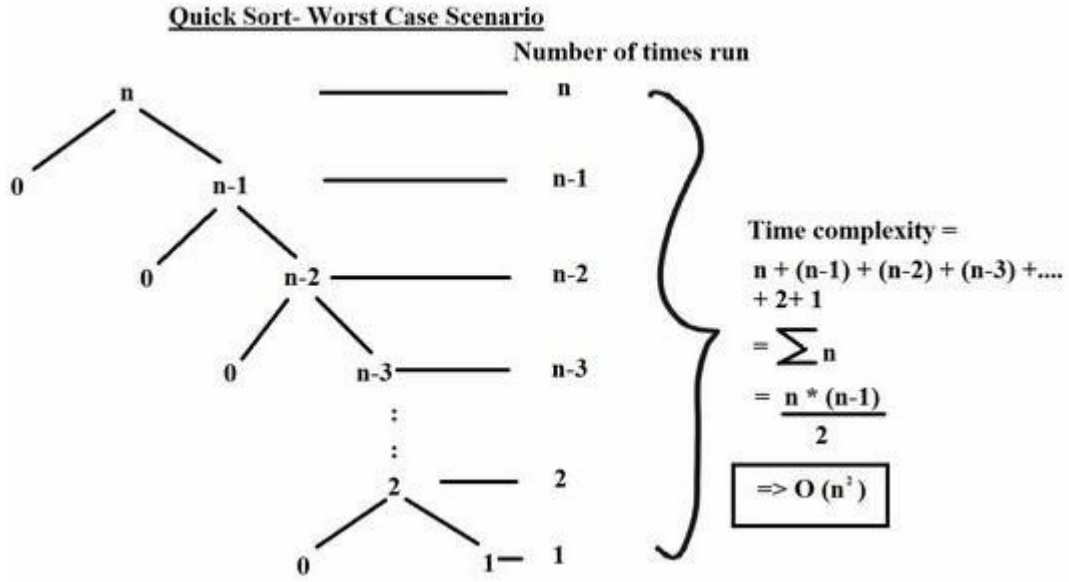
Example:



```
/* low --> Starting index, high --> Ending index */
quickSort(arr[], low, high)
{
    if (low < high)
    {
        /* pi is partitioning index, arr[pi] is now at right place */
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // Before pi
        quickSort(arr, pi + 1, high); // After pi
    }
}
/* This function takes last element as pivot, places
   the pivot element at its correct position in sorted
   array, and places all smaller (smaller than pivot)
   to left of pivot and all greater elements to right
   of pivot */
partition (arr[], low, high)
{
    // pivot (Element to be placed at right position)
    pivot = arr[high];

    i = (low - 1) // Index of smaller element and indicates the
                  // right position of pivot found so far
    for (j = low; j <= high- 1; j++)
    {
        // If current element is smaller than the pivot
        if (arr[j] < pivot)
        {
            i++;    // increment index of smaller element
            swap arr[i] and arr[j]
        }
    }
    swap arr[i + 1] and arr[high])
    return (i + 1)
}
```





Implementation:

Conclusion: Comment on the implementation of Quick Sort Algorithm.

Experiment No. 4
Fractional Knapsack using Greedy Method
Date of Performance:
Date of Submission:

Experiment No. 4

Title: Fraction Knapsack

Aim: To study and implement Fraction Knapsack Algorithm

Objective: To introduce Greedy based algorithms

Theory:

Greedy method or technique is used to solve Optimization problems. A solution that can be maximized or minimized is called Optimal Solution.

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a mass and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible. It derives its name from the problem faced by someone who is constrained by a fixed size knapsack and must fill it with the most valuable items. The most common problem being solved is the 0-1 knapsack problem, which restricts the number xi of copies of each kind of item to zero or one.

In Knapsack problem we are given:1) n objects 2) Knapsack with capacity m, 3) An object i is associated with profit Wi , 4) An object i is associated with profit Pi , 5) when an object i is placed in knapsack we get profit Pi Xi .

Here objects can be broken into pieces (Xi Values) The Objective of Knapsack problem is to maximize the profit.

Example:

In this version of Knapsack problem, items can be broken into smaller pieces. So, the thief may take only a fraction x_i of ith item.

$$0 \leqslant xi \leqslant 1$$

The ith item contributes the weight xi.wi to the total weight in the knapsack and profit xi.pi to the total profit**.**

```
greedy_fractional_knapsack (w[1...n], p[1...n], W)
for i=1 to n
    do x[i] = 0
weight = 0
for i=1 to n
    if weight + w[i] ≤ W then
        x[i] = 1
        weight = weight + w[i]
    else
        x[i] = (W-weight) / w[i]
        weight = W
        break
return x
```

i=1 → B

0+10 ≤ 60

x[i] = 1

wt = 10

i=2 → A

10+40

50 ≤ 60

x[i] = 2

10+40

wt = 50

i=3 → C

(60-50)/20

x[i] = 10/20 = 1/2

wt = 60

x = [A, B, 1/2 C]

Total wt

10+40+20*(10/20)

= 60

~~x[i] = 0~~

~~wt = 0~~

Total profit is

100+280+120*(10/20)

280+60 = 440

Ex:

W = 60

Item	A	B	C	D
profit	280	100	120	120
weight	40	10	20	24
Ratio $\left(\frac{P_i}{w_i}\right)$	7	10	6	5

provided items are not sorted based on $\frac{P_i}{w_i}$.

sorted

Item	B	A	C	D
profit	100	280	120	120
weight	10	40	20	24
Ratio $\left(\frac{P_i}{w_i}\right)$	10	7	6	5

Algorithm:

Hence, the objective of this algorithm is to

$$maximize \sum_{n=1}^{n}(x_i . pi)$$

subject to constraint,

$$\sum_{n=1}^{n}(x_i . wi) \leqslant W$$

It is clear that an optimal solution must fill the knapsack exactly, otherwise we could add a fraction of one of the remaining items and increase the overall profit.

Thus, an optimal solution can be obtained by

$$\boxed{\sum_{n=1}^{n}(x_i . wi) = W}$$

In this context, first we need to sort those items according to the value of $\frac{p_i}{w_i}$, so that $\frac{p_{i+1}}{w_{i+1}} \leq \frac{p_i}{w_i}$. Here, ***x*** is an array to store the fraction of items.

```
Algorithm: Greedy-Fractional-Knapsack (w[1..n], p[1..n], W)
for i = 1 to n
   do x[i] = 0
weight = 0
for i = 1 to n
   if weight + w[i] ≤ W then
      x[i] = 1
      weight = weight + w[i]
   else
      x[i] = (W - weight) / w[i]
      weight = W
      break
return x
```

Implementation:

Conclusion: Fractional Knapsack algorithm has been successfully implemented.

Experiment No. 5
Prim's Algorithm
Date of Performance:
Date of Submission:

Experiment No. 5

Title: Prim's Algorithm and Kruskal's Algorithm

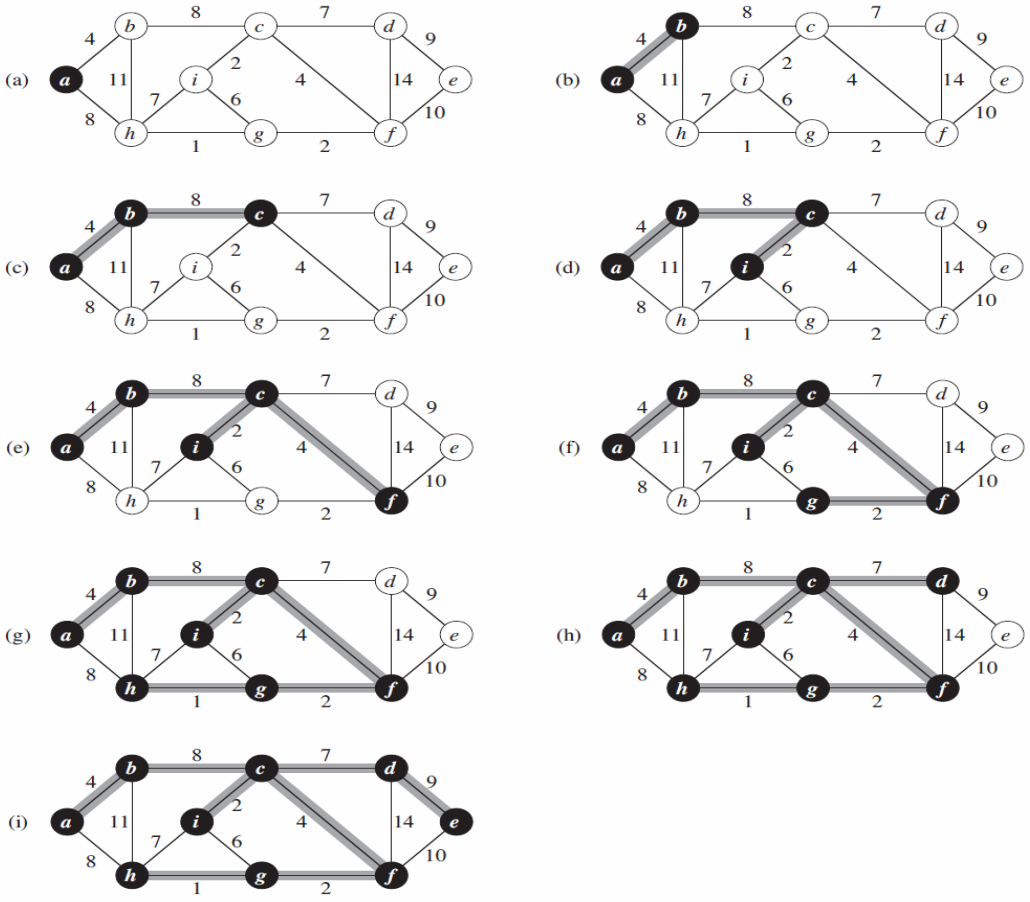
Aim: To study and implement Prim's and Kruskal's Minimum Cost Spanning TreeAlgorithm

Objective: To introduce Greedy based algorithms.

Theory:

Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a weighted undirected graph. This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized. The algorithm operates by building this tree one vertex at a time, from an arbitrary starting vertex, at each step adding the cheapest possible connection from the tree to another vertex.

Example:



Algorithm and Complexity:

```
1   Algorithm Prim(E, cost, n, t)
2   // E is the set of edges in G. cost[1 : n, 1 : n] is the cost
3   // adjacency matrix of an n vertex graph such that cost[i, j] is
4   // either a positive real number or ∞ if no edge (i, j) exists.
5   // A minimum spanning tree is computed and stored as a set of
6   // edges in the array t[1 : n − 1, 1 : 2]. (t[i, 1], t[i, 2]) is an edge in
7   // the minimum-cost spanning tree. The final cost is returned.
8   {
9       Let (k, l) be an edge of minimum cost in E;
10      mincost := cost[k, l];
11      t[1, 1] := k; t[1, 2] := l;
12      for i := 1 to n do  // Initialize near.
13          if (cost[i, l] < cost[i, k]) then near[i] := l;
14          else near[i] := k;
15      near[k] := near[l] := 0;
16      for i := 2 to n − 1 do
17      { // Find n − 2 additional edges for t.
18          Let j be an index such that near[j] ≠ 0 and
19          cost[j, near[j]] is minimum;
20          t[i, 1] := j; t[i, 2] := near[j];
21          mincost := mincost + cost[j, near[j]];
22          near[j] := 0;
23          for k := 1 to n do // Update near[ ].
24              if ((near[k] ≠ 0) and (cost[k, near[k]] > cost[k, j]))
25                  then near[k] := j;
26      }
27      return mincost;
28  }
```

Time Complexity is O(n2), Where, n = number of vertices **Theory:**

Implementation:

Conclusion: Comment on implementation of Prim's Algorithm.

Experiment No. 6
All Pairs Shortest Path Algorithm
Date of Performance:
Date of Submission:

Experiment No. 6

Title: All Pair Shortest path, Dynamic Programming: Floyd Warshall

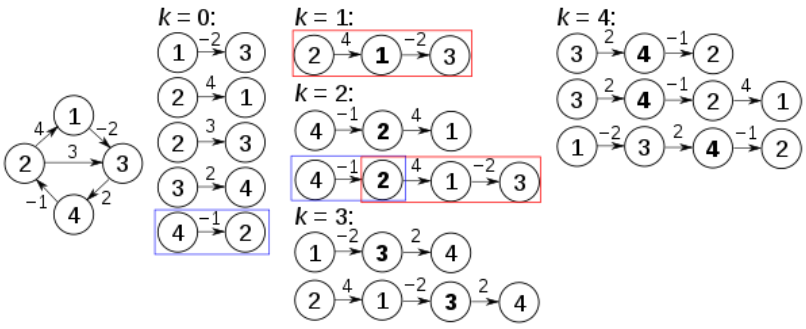
Aim: To study and implement All Pair Shortest path, Dynamic Programming: Floyd Warshall.

Objective: To introduce Dynamic Programming Approaches

Theory:

the Floyd–Warshall algorithm is an algorithm for finding shortest paths in a directed weighted graph with positive or negative edge weights (but with no negative cycles). A single execution of the algorithm will find the lengths (summed weights) of shortest paths between all pairs of vertices. Although it does not return details of the paths themselves, it is possible to reconstruct the paths with simple modifications to the algorithm. Versions of the algorithm can also be used for finding the transitive closure of a relation 'R', or (in connection with the Schulze voting system) widest paths between all pairs of vertices in a weighted graph.

Example:



Algorithm:

```
FLOYD-WARSHALL(W)
1. n ← rows[W]
2. D^(0) ← W
3. for k ← 1 to n
4.     do for i ← 1 to n
5.         do for j ← 1 to n
6.             d_ij^(k) ← min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
7. return D^(n)
```

Implementation:

Conclusion: Comment on the implementation of Floyd Warshall Algorithm.

Experiment No. 7
Sum of Subset Algorithm
Date of Performance:
Date of Submission:

Experiment No: 7

Title: Sum of Subset

Aim: To study and implement Sum of Subset problem

Objective: To introduce Backtracking methods

Theory:

Backtracking is finding the solution of a problem whereby the solution depends on the previous steps taken. For example, in a maze problem, the solution depends on all the steps you take one-by-one. If any of those steps is wrong, then it will not lead us to the solution. In a maze problem, we first choose a path and continue moving along it. But once we understand that the particular path is incorrect, then we just come back and change it. This is what backtracking basically is.

In backtracking, we first take a step and then we see if this step taken is correct or not i.e., whether it will give a correct answer or not. And if it doesn't, then we just come back and change our first step. In general, this is accomplished by recursion. Thus, in backtracking, we first start with a partial sub-solution of the problem (which may or may not lead us to the solution) and then check if we can proceed further with this sub-solution or not. If not, then we just come back and change it.

Thus, the general steps of backtracking are:

- Start with a sub-solution
- check if this sub-solution will lead to the solution or not
- If not, then come back and change the sub-solution and continue again.

The subset sum problem is a classic optimization problem that involves finding a subset of a given set of positive integers whose sum matches a given target value. More formally, given a set of non-negative integers and a target sum, we aim to determine whether there exists a subset of the integers whose sum equals the target.

Let's consider an example to better understand the problem. Suppose we have a set of integers [1, 4, 6, 8, 2] and a target sum of 9. We need to determine whether there exists a subset within the given set whose sum equals the target, in this case, 9. In this example, the subset [1, 8] satisfies the condition, as their sum is indeed 9.

Solving Subset Sum with Backtracking

To solve the subset, sum problem using backtracking, we will follow a recursive approach. Here's an outline of the algorithm:

1. Sort the given set of integers in non-decreasing order.
2. Start with an empty subset and initialize the current sum as 0.
3. Iterate through each integer in the set:
   - Include the current integer in the subset.
   - Increment the current sum by the value of the current integer.
   - Recursively call the algorithm with the updated subset and current sum.
   - If the current sum equals the target sum, we have found a valid subset.
   - Backtrack by excluding the current integer from the subset.
   - Decrement the current sum by the value of the current integer.

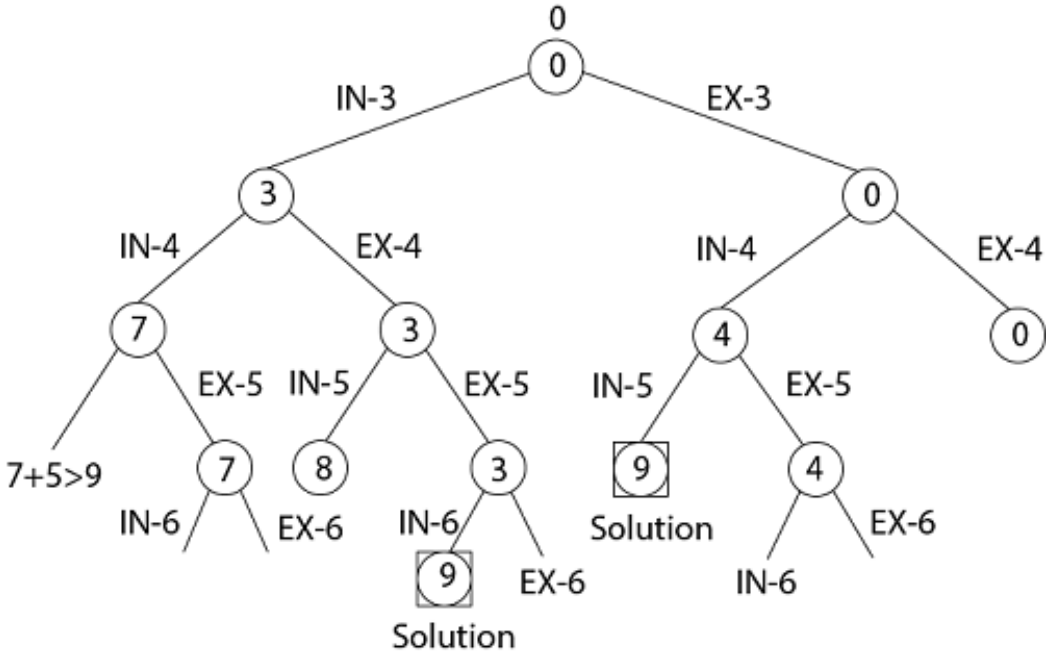
4.If we have exhausted all the integers and none of the subsets sum up to the target, we conclude that there is no valid subset.

Example:

Given a set S = (3, 4, 5, 6) and X =9. Obtain the subset sum using the backtracking approach.

Initially S = (3, 4, 5, 6) and X =9.

S'= (∅)



Implementation:

Conclusion: The Sum of Subset problem has been implemented.

Experiment No. 8
N-Queen Algorithm
Date of Performance:
Date of Submission:

Experiment No. 8

Title: N-Queen

Aim: To study and implement N-Queen

Objective: To introduce Backtracking and Branch-Bound methods

Theory:

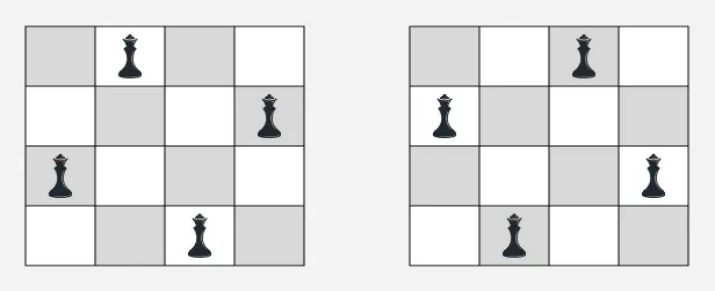
Given an integer n, place n queens on an n × n chessboard such that no two queens attack each other. A queen can attack another queen if they are placed in the same row, the same column, or on the same diagonal. For example, following is a solution for 4 Queen problem.

Example:

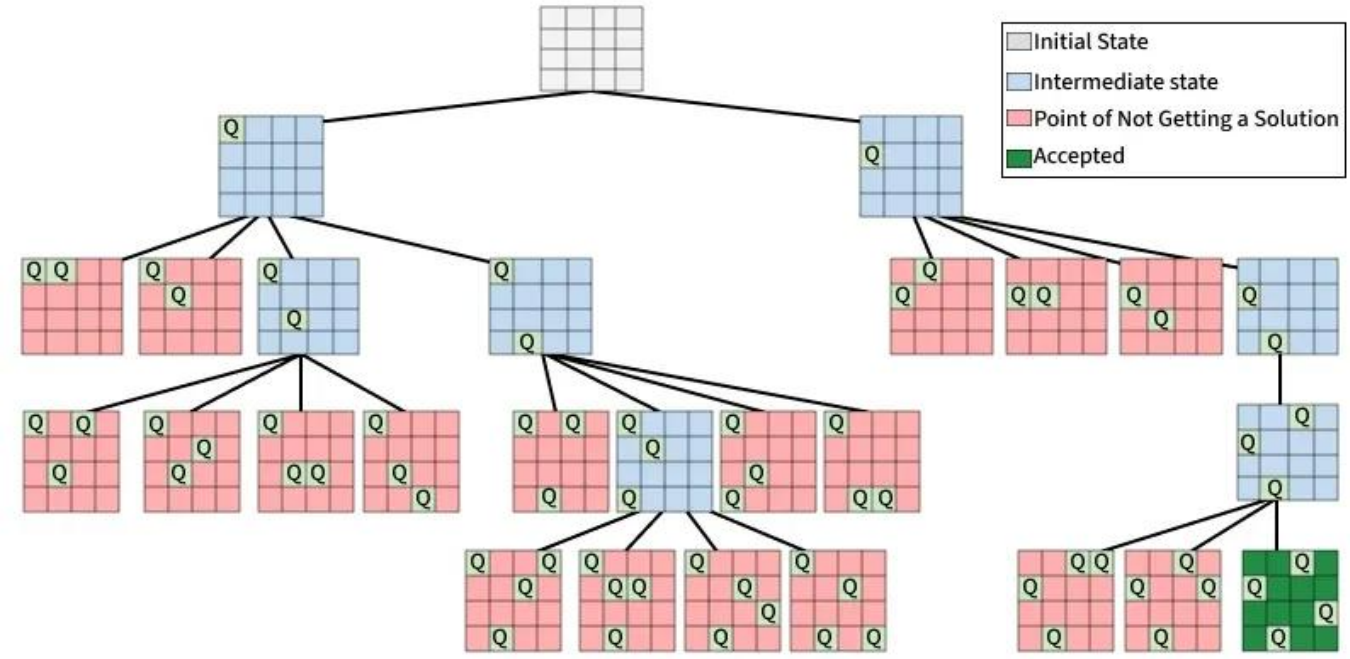
Input: n = 4

Output: [[2, 4, 1, 3], [3, 1, 4, 2]]

Explanation: Below is Solution for 4 queen problem



State Space Tree:



Implementation:

Conclusion: The n-queen problem has been implemented.

Experiment No. 9
Rabin-Karp Algorithm
Date of Performance:
Date of Submission:

Experiment No. 9

Title: Implementation of Rabin-Karp Algorithm for String Pattern Matching

Aim: To implement the Rabin-Karp algorithm to efficiently search for a pattern in a given text using hashing.

Objective:

- To understand the concept of string pattern matching.
- To learn the working of the Rabin-Karp algorithm using hash values.
- To compare the hash value of the pattern with the hash values of text substrings.

Theory:

The **Rabin-Karp algorithm** is a **string matching algorithm** proposed by **Michael O. Rabin** and **Richard M. Karp** in 1987. It is based on the **hashing technique**, where the pattern and each substring of the text having the same length as the pattern are converted into numerical hash values.

Instead of comparing characters one by one for every position, the algorithm first compares the **hash values**:

- If the hash values are different, the pattern cannot match.
- If the hash values are the same, a character-by-character comparison is performed to confirm the match (to avoid false matches due to hash collisions).

To improve efficiency, the algorithm uses a **rolling hash**, which computes the next substring's hash value from the previous one without recalculating it from scratch.

Example:

Text
AABAACAADAABAABA
Pattern
AABA
Step 1: Compute Hash Values

- Hash(Pattern) = H(P)

- Hash(First Window) = $H(T_1)$

If:

$H(P) = H(T_1)$

Perform character comparison.

A A B A

A A B A

Match found at Index 0

Slide the Window

Next substring:

ABAA

Compute its new hash using the rolling hash.

If the hash values are different:

$H(P) \neq H(T_2)$

No comparison is needed.

Continue sliding the window until the end of the text.

Output

Pattern found at index 0

Pattern found at index 9

Pattern found at index 12

Algorithm:

Input: Text T of length n, Pattern P of length m

Output: Starting index of each occurrence of the pattern

1. Start.
2. Read the text T and pattern P.
3. Compute the hash value of the pattern.
4. Compute the hash value of the first substring (window) of the text of length m.
5. For each window from position 0 to n − m:
    - If the hash values are equal:
        - Compare the pattern and the current text window character by character.
        - If all characters match, report the current index.
    - If there are more windows:
        - Update the text window hash using the rolling hash technique.

6. Repeat until all windows are checked.
7. Stop.

Implementation:

Conclusion: Comment on the implementation of the Rabin-Karp algorithm.

Experiment No. 10
Min-Max Algorithm
Date of Performance:
Date of Submission:

Experiment No. 10

Title: Implementation of Min-Max Algorithm using Divide and Conquer

Aim: To implement the Divide and Conquer (DAC) approach to find the minimum and maximum elements in an array efficiently.

Objective:

- To understand the Divide and Conquer paradigm.
- To find both the minimum and maximum elements using fewer comparisons.

Theory:

The **Min-Max algorithm using Divide and Conquer (DAC)** is an efficient method to find both the **minimum** and **maximum** elements in an array simultaneously.

Instead of scanning the array sequentially, the algorithm **divides** the array into two halves, recursively finds the minimum and maximum in each half, and then **combines** the results.

The Divide and Conquer technique consists of three steps:

1. **Divide:** Split the array into two equal halves.
2. **Conquer:** Recursively find the minimum and maximum in each half.
3. **Combine:** Compare the minimum values of both halves to obtain the overall minimum and compare the maximum values to obtain the overall maximum.

Example:

Input Array

[15, 8, 20, 5, 12, 30, 2, 18]

Step 1: Divide

[15, 8, 20, 5, 12, 30, 2, 18]

/ \

[15, 8, 20, 5] [12, 30, 2, 18]

Step 2: Further Divide

[15, 8] [20, 5] [12, 30] [2, 18]

Step 3: Base Cases

[15,8] → Min = 8, Max = 15

[20,5] → Min = 5, Max = 20

[12,30] → Min = 12, Max = 30

[2,18] → Min = 2, Max = 18

Step 4: Combine Results

For Left Half:

Min = min(8,5) = 5

Max = max(15,20) = 20

For Right Half:

Min = min(12,2) = 2

Max = max(30,18) = 30

Final Comparison

Overall Min = min(5,2) = 2

Overall Max = max(20,30) = 30

Output

Minimum Element = 2

Maximum Element = 30

Algorithm:

```
MinMax(A, low, high)

If low == high
   min = max = A[low]

Else if high == low + 1
   If A[low] < A[high]
```

```
    min = A[low]
    max = A[high]
  Else
    min = A[high]
    max = A[low]

Else
  mid = (low + high) / 2

  Find MinMax(A, low, mid)

  Find MinMax(A, mid + 1, high)

  min = minimum(left.min, right.min)

  max = maximum(left.max, right.max)

Return(min, max)
```

Implementation:

Conclusion: Comment on the implementation of the Max-Min Algorithm